Ben-Gurion University of the Negev
Faculty of Natural Sciences
Department of Computer Science

Bitcoin Wallet Risk Analyzer

Detecting Criminal Bitcoin Wallets with Graph Neural Networks

מנתח סיכון לארנקי ביטקוין — זיהוי ארנקים פליליים באמצעות רשתות נוירונים גרפיות

Project Team

Ori Sinvani (325770824)

Harel Brener (214179012)

Nehoray Chalfon (325833531)

Project Advisor

Ofer Zevin

Project Year: 2025–2026 (תשפ"ו)

Code repository: github.com/NehorayChalfon0166/final_project

Live demo: cryptotrace.cs.bgu.ac.il

Abstract

Cryptocurrency networks are increasingly exploited for money laundering, ransomware payments, fraud, and terror financing. Because Bitcoin is pseudonymous and its transaction graph is massive and dynamic, the compliance teams at banks, exchanges, and law-enforcement agencies still rely mostly on static blacklists that only flag wallets *after* they are already known to be malicious. This project asks a data-science question instead: given only a Bitcoin address and its public transaction history, can we decide whether the wallet has been used for criminal activity?

We frame the problem as binary node classification over per-wallet ego-graphs and train a three-layer **GATv2** graph attention network that reads both a wallet's own behaviour and the pattern of its one-hop transactions. The model is trained on a balanced merge of two public labelled datasets, **REAL-CATS** (modern criminal wallets) and **Elliptic++** (theft/Ponzi era), comprising 86,870 training wallets and 21,718 held-out test wallets. On the held-out test set the GNN reaches an **F_1 of 0.869**, accuracy 0.894, recall 0.884, and **ROC-AUC 0.959**, with well-calibrated probabilities after temperature scaling. It beats an XGBoost classifier trained on the same twelve tabular features (F_1 0.755) and a plain GCN baseline (F_1 0.582). The $\sim$11-point F_1 gap over XGBoost is the central finding: the local transaction structure around a wallet carries information that the wallet's own summary statistics do not. The trained model is served through a FastAPI backend and a React web application deployed at cryptotrace.cs.bgu.ac.il.

תקציר

רשתות מטבעות קריפטוגרפיים מנוצלות יותר ויותר להלבנת הון, תשלומי כופרה, הונאה ומימון טרור. מכיוון שביטקוין הוא פסאודו-אנונימי וגרף העסקאות שלו עצום ודינמי, צוותי הרגולציה בבנקים, בבורסות ובגופי אכיפת החוק עדיין מסתמכים בעיקר על רשימות שחורות סטטיות המסמנות ארנקים רק לאחר שכבר ידוע שהם זדוניים. פרויקט זה שואל שאלה מדעית-נתונים חלופית: בהינתן כתובת ביטקוין והיסטוריית העסקאות הציבורית שלה בלבד, האם ניתן להכריע אם הארנק שימש לפעילות פלילית?

הגדרנו את הבעיה כסיווג בינארי של צמתים מעל גרפי-אֶגו של ארנקים בודדים, ואימנו רשת קשב גרפית מסוג GATv2 בת שלוש שכבות הקוראת גם את התנהגות הארנק עצמו וגם את דפוס העסקאות של שכניו הישירים. המודל אומן על מיזוג מאוזן של שני מאגרי נתונים ציבוריים מתויגים: REAL-CATS (ארנקים פליליים עדכניים) ו-Elliptic++, בסך 86,870 ארנקים לאימון ו-21,718 ארנקים למבחן. על קבוצת המבחן השיג המודל ציון F1 של 0.869, דיוק 0.894, היזכרות 0.884 ו-ROC-AUC של 0.959, עם הסתברויות מכוילות היטב לאחר כיול טמפרטורה. המודל עקף גם מסווג XGBoost שאומן על אותם

שנים-עשר מאפיינים טבלאיים (F1 של 0.755) וגם בסיס GCN פשוט (F1 של 0.582).
הפער של כ-11 נקודות ב-F1 על פני XGBoost הוא הממצא המרכזי: המבנה המקומי של
העסקאות סביב הארנק נושא מידע שאין במאפייני הארנק עצמו. המודל המאומן מוגש
דרך שרת FastAPI ואפליקציית React שנפרסו בכתובת cryptotrace.cs.bgu.ac.il.

Contents

List of Figures

List of Tables

1. Introduction

1.1 Background and Motivation

Bitcoin transactions are recorded on a public ledger, yet the identities behind wallet addresses are not. This pseudonymity is exactly what makes the network attractive for illicit use: ransomware operators, darknet markets, scams, and sanctioned entities move funds through webs of addresses designed to obscure the origin of money. Regulated institutions — banks and cryptocurrency exchanges — are legally required to screen the wallets they interact with, and security agencies must trace illicit funding flows. In practice, most of this screening is still done by copying an address into a static database and checking whether it has *already* been blacklisted. Such rule-based workflows are reactive: they cannot flag a freshly created wallet that is structurally embedded in a criminal network but not yet reported. At the scale and complexity of the Bitcoin graph, manual analysis does not keep up.

1.2 The Gap

Prior academic work has shown that Graph Neural Networks (GNNs) are effective at detecting illicit Bitcoin activity, but most studies train and evaluate on a *single* source — overwhelmingly the Elliptic dataset — and therefore optimise for one historical crime pattern (2016-era theft and Ponzi schemes). It remains open whether the structural signatures they learn transfer to modern, distinct threats such as extortion, scams, and ransomware. The gap this project targets is **cross-dataset generalisation**: whether illicit activity carries a structural signature consistent enough that a single model, trained on two datasets covering different crime types and time periods, can classify wallets it has never seen.

1.3 Proposed Method (Overview)

We treat the task as binary classification of a wallet (criminal vs. benign) from a small graph built around it. Each wallet becomes the centre of an *ego-graph*: the wallet itself carries twelve behavioural features, every counter-party it transacted with becomes a neighbour node, and each interaction becomes a directed, weighted edge. A three-layer GATv2 graph attention network aggregates this neighbourhood into a single risk score. We merge REAL-CATS and Elliptic++ into one balanced corpus, and compare our model against two baselines that deliberately remove one ingredient each: an XGBoost classifier that sees the wallet's own features but no graph, and a plain GCN that sees the graph but ignores edge detail. Figure 1.1 shows the end-to-end flow, from an input address to a calibrated verdict.

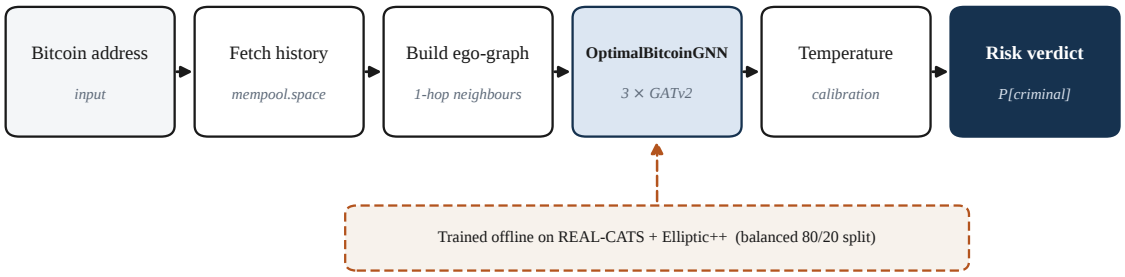

Figure 1.1. End-to-end method. A Bitcoin address is expanded into a one-hop ego-graph from live `mempool.space` data, passed through the trained GATv2 model, and returned as a calibrated risk verdict. The same model is trained offline on the balanced merge of REAL-CATS and Elliptic++.

1.4 Main Findings

On a held-out test set of 21,718 wallets, the GNN reaches $F_1 = 0.869$ and ROC-AUC $= 0.959$, clearly ahead of XGBoost ($F_1 = 0.755$) and the plain GCN ($F_1 = 0.582$). The two controlled contrasts localise *why*: the jump over the plain GCN shows that per-transaction detail (amount, direction, timing) matters, and the jump over XGBoost shows that the one-hop neighbourhood adds information beyond the wallet's own statistics. After temperature scaling the probabilities are well calibrated, so the risk score shown to a user can be read at face value.

1.5 Document Structure

Chapter 2 states the project goals. Chapter 3 reviews related work. Chapter 4 describes the two datasets and the unified feature schema. Chapter 5 covers exploratory analysis and preprocessing. Chapter 6 details the data split, the ego-graph construction, and the three models. Chapter 7 defines the experimental protocol and metrics. Chapter 8 presents results and visualisations. Chapter 9 concludes with limitations and future work. Appendices cover the deployed system's characterisation and testing, and full reproducibility details.

2. Project Goals

The project has one primary goal and several concrete objectives that make it measurable.

Primary goal. Given a Bitcoin wallet address and its public transaction history, automatically classify the wallet as *criminal* or *benign* and return a calibrated risk score.

G1 — Graph model.
Build a GNN that classifies a wallet from a one-hop ego-graph combining the wallet's behaviour with its transaction neighbourhood.

G2 — Unified, balanced corpus.
Merge REAL-CATS and Elliptic++ onto a common feature schema, remove overlap and label conflicts, and produce a class-balanced, source-stratified train/test split.

G3 — Cross-dataset generalisation.
Test whether one model trained across both sources generalises across crime types and time periods, rather than over-fitting one source.

G4 — Rigorous evaluation.
Quantify performance against a tabular baseline (XGBoost) and a graph baseline (plain GCN) on identical wallets, using discrimination, per-class, and calibration metrics.

G5 — Usable, calibrated product.
Serve the model as a live web application whose displayed confidence values are trustworthy (post-hoc calibration).

3. Literature Review

Cryptocurrency forensics has moved from heuristic rules to deep learning, and because blockchain data is naturally a graph of wallets and transactions, GNNs have become the dominant methodology. This review focuses on the strands most relevant to our design: node/transaction-level GNNs, structural and subgraph analysis, temporal and imbalance-aware methods, and the datasets that anchor the field.

3.1 GNNs for Fraud Detection

The foundational approach classifies individual wallets or transactions in isolation. Asiri et al. [2] show that optimised GCNs reach high accuracy on the Elliptic dataset when structural features are combined with cost-sensitive training, validating GCN as a strong baseline. Zhang et al. [15] introduce GAT-ResNet, using graph *attention* to weigh the importance of neighbours and thereby handle noisy blockchain data better than plain GCNs. This directly motivates our choice of a GATv2 attention backbone [3, 13] over a vanilla GCN [8].

3.2 Structural and Subgraph Analysis

Node-level models often miss sophisticated actors who launder through long chains of intermediary wallets. Ouyang et al. [9] argue that laundering schemes such as "peel chains" appear as distinct geometric motifs in the graph rather than as isolated bad nodes, and that analysing the surrounding subgraph captures operations that node-centric models miss. This supports our decision to classify a wallet from its neighbourhood rather than from its features alone.

3.3 Temporal Behaviour and Class Imbalance

Two operational hurdles are the temporal nature of the data and the scarcity of illicit labels. Alarab and Prakoonwit [1] combine GCNs with an LSTM to model how transaction behaviour evolves over time and address the cold-start problem via active learning. The RL-GNN fusion line of work [10] tackles real-time detection in imbalanced streams by dynamically adjusting fraud thresholds to cope with concept drift. We do not model time recurrently, but we adopt imbalance-aware practice by *balancing* the training corpus and encoding a normalised timestamp on every edge.

3.4 Datasets and Benchmarking

High-quality labelled data is the prerequisite for all of the above. The Elliptic dataset [14] and its extension Elliptic++ [5] are the standard benchmarks but focus on 2014–2017 theft/Ponzi activity. REAL-CATS [12] expands coverage into modern threats (scams, ransomware, extortion, darknet markets). Using both is central to our generalisation hypothesis.

3.5 Positioning of This Project

Most reviewed studies train and test on a single source and therefore learn source-specific patterns. Our contribution is to build a *unified* model across Elliptic++ (theft/Ponzi) and REAL-CATS (modern cybercrime), testing whether illicit activity carries a structural signature robust across crime types and time periods — a question the prior single-dataset work leaves open.

4. Data Description

4.1 Sources

REAL-CATS [12] is an expert-curated dataset of Bitcoin wallets linked to criminal activity, compiled by domain analysts from public criminal-wallet reports (sanctions lists, ransomware disclosures, law-enforcement notices), paired with a benign control set. Its labels come from human reporting rather than on-chain inference; the material was collected through roughly 2023–2024. **Elliptic++** [5] extends the original Elliptic dataset [14]; its transactions span roughly 2014–2017, organised into 49 time-steps of about two weeks each.

4.2 Unified Feature Schema

Both datasets are mapped onto a common schema of nineteen features computable from either source: eleven shared base features (total transactions, total sent and received, total fees, lifetime, send and receive counts, max/min sent and received), three features derivable from both (mean blocks between transactions, mean fee share, average transaction size), and five engineered features (send/receive ratio, activity rate, fee per transaction, transaction-size range, in/out balance). Every feature is also stored as its $\log(1 + x)$ transform. After correlation pruning — dropping seven features that were highly correlated with another ($r > 0.95$) — the model uses the twelve least-redundant log-scaled features listed in Table 4.1.

Table 4.1. The twelve log-scaled features used by the model (after correlation pruning).

Feature	Meaning
`fee_per_tx_log`	Average fee paid per transaction
`fee_share_mean_log`	Mean fraction of transaction value spent on fees
`tx_size_range_log`	Spread between largest and smallest sent amount
`total_txs_log`	Lifetime number of transactions
`send_receive_ratio_log`	Ratio of outgoing to incoming activity
`avg_tx_size_log`	Average sent amount
`max_sent_log`	Largest single amount sent
`in_out_balance_log`	Balance between received and sent volume
`max_received_log`	Largest single amount received
`blocks_btwn_txs_mean_log`	Mean number of blocks between transactions
`activity_rate_log`	Transactions per unit lifetime
`lifetime_seconds_log`	Time between first and last transaction

4.3 Balanced Split

A balancing step drops cross-source addresses whose labels disagree, collapses agreeing duplicates, keeps every criminal wallet plus a matched random sample of benign wallets within each source, and produces a stratified 80/20 split keyed on the joint (label, source). Both classes and both sources appear in identical proportions in train and test (Table 4.2); the random seed is fixed throughout.

Table 4.2. Balanced, source-stratified train/test split.

Split	Rows	Criminal	Benign	REAL-CATS	Elliptic++
Training set	86,870	43,435	43,435	64,044	22,826
Test set	21,718	10,859	10,859	16,012	5,706

5. Data Exploration and Pre-processing

5.1 Exploratory Analysis

Exploratory analysis of both datasets guided every downstream choice. In **Elliptic++**, labels are heavily imbalanced (illicit transactions average $\sim$11% and only a small fraction of wallets are labelled), and illicit nodes are typically low-degree while licit nodes form high-degree hubs — illicit actors rarely form tightly connected clusters and instead blend into the wider network by transacting with unknown or licit nodes. Value distributions differ by class: illicit transactions concentrate at lower values with a sharper peak, whereas licit transactions have a heavier right tail. In **REAL-CATS**, criminal wallets are more active than benign ones (median transaction count 4 vs. 2) and move more money (median total sent \$2,426 vs. \$1,049), which confirms that activity- and volume-based features are discriminative. The criminal category mix is dominated by scams and ransomware, with darknet markets and mixing services rarer but showing the highest transaction intensity.

5.2 Pre-processing Pipeline

The findings above translate into a fixed pre-processing pipeline, applied identically at training and at inference:

1. **Cleaning.** Remove zero-transaction addresses; resolve satoshi-to-BTC unit scaling so amounts are on a consistent scale.
2. **Feature engineering.** Compute the nineteen-feature schema, including the five engineered ratios, from raw transaction records.
3. **Log transform.** Apply $\log(1 + x)$ to every feature to compress heavy tails seen in the EDA.
4. **Clipping.** Clip engineered features to training-derived ranges (activity rate to $[0, 1000]$; send/receive ratio and in/out balance to $[0, 100]$; fee share to $[0, 1]$) so a live wallet cannot produce out-of-distribution values.
5. **Correlation pruning.** Drop the seven features with pairwise $r > 0.95$, leaving the twelve features of Table 4.1.
6. **Balancing & split.** Balance classes within each source and take the stratified 80/20 split of Table 4.2.

Maintaining *train/inference parity* — running exactly this pipeline on live wallet data — was essential: an early unit mismatch (satoshis vs. BTC) made inference features up to $90{,}000\times$

larger than training features and skewed predictions until the scaling and clipping were made identical on both sides.

6. Data Split, Ego-Graphs, and Models

6.1 Per-Wallet Ego-Graphs

For each wallet a small graph is built. The wallet sits at the centre carrying the twelve log features. Every counter-party it has transacted with appears as a *ghost* node; at forward time the model replaces each ghost's zero-feature placeholder with a learned embedding plus a node-type embedding, so ghost neighbours are not treated as fresh wallets with all-zero behaviour. Each interaction is a directed edge with three features: the log of the amount transferred, a direction flag (1 outgoing, 0 incoming), and a timestamp normalised against the Bitcoin genesis block. Transaction history is fetched from the public `mempool.space` API and cached per address.

6.2 The Model: OptimalBitcoinGNN

The main model is a three-layer GATv2 graph attention network [3, 6] with $4 \to 4 \to 2$ attention heads, hidden size 64, and an edge dimension of 3. It uses residual connections from the input through the GNN layers, DropEdge regularisation [11], and a hybrid readout that concatenates the centre-node embedding with global mean-pool and global max-pool ($3 \times 64 = 192$), followed by a $192 \to 64 \to 2$ classifier. Figure 6.1 shows the architecture.

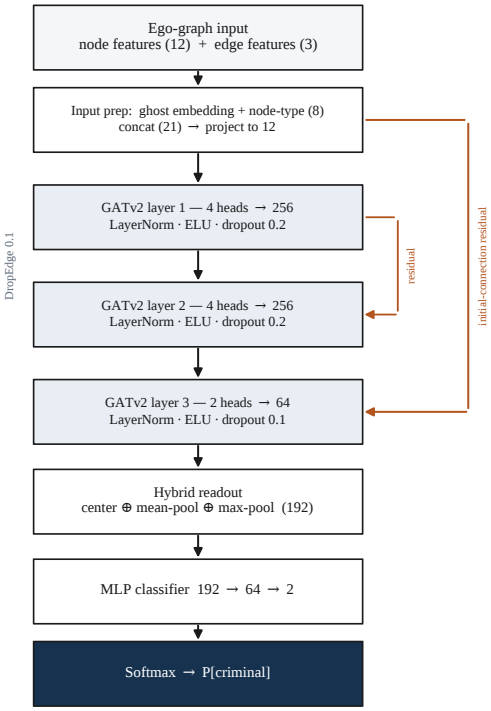

Figure 6.1. OptimalBitcoinGNN architecture: three GATv2 layers with multi-head attention, learnable ghost/node-type embeddings, an initial-connection residual, and a hybrid (centre $\oplus$ mean-pool $\oplus$ max-pool) readout feeding a two-layer classifier.

6.3 Baselines

Two baselines each remove exactly one ingredient, so the comparisons are controlled:

- **XGBoost (tabular)** [4]: trained on the same twelve features and the same split, but with *no* graph. The contrast with the GNN isolates the value of the one-hop neighbourhood.
- **Basic GCN (graph)** [8]: a vanilla GCN on the same ego-graphs but without learnable ghost embeddings, without the input-to-final residual, and with a plain mean-pool readout instead of the hybrid readout. The contrast isolates the value of our architectural additions and of per-edge transaction detail.

6.4 Calibration

Neural classifiers are often over-confident [7]. After training, a single temperature parameter is fit by LBFGS on a held-out calibration slice and applied at inference in the backend, so the displayed probabilities can be read as honest confidence values.

7. Experiments

7.1 Protocol

The 80/20 stratified split gives the canonical train and test sets. Inside training, the training pool is further split 85/15 (fixed seed) into train and validation, the latter used for early stopping and best-epoch selection on validation F_1. A small slice is reserved *before* that split, exclusively for temperature scaling. All three models are trained and evaluated on *exactly the same wallets* — the rows whose address has a cached ego-graph — so the comparison is row-for-row fair; XGBoost is retrained on the same intersection rather than on the full CSV.

7.2 Measurement Method and Metrics

The primary metric is F_1 on the held-out test split, complemented by a standard battery so a single number cannot hide the full picture (Table 7.1). Model selection during training uses validation F_1 with early stopping (patience 20). The evaluation step also dumps misclassified test addresses (false positives and false negatives) to CSV and writes per-feature gradient-saliency scores to JSON.

Table 7.1. Evaluation metrics.

Family	Metrics
Discrimination	Accuracy, Precision, Recall, F_1, ROC-AUC, PR-AUC
Per-class	Precision / Recall / F_1 for the benign and criminal classes
Confusion	Confusion matrix on the test set
Calibration	Brier score, Expected Calibration Error, reliability diagram

7.3 Key Hyperparameters

Table 7.2 lists the settings that reproduce the reported numbers; the full set is in Appendix A.5.

Table 7.2. Key hyperparameters.

Component	**Setting**
GNN	$3\times$ GATv2Conv (4/4/2 heads), hidden 64, edge dim 3, dropout 0.2 (final 0.3), DropEdge 0.1
Readout	centre $\oplus$ mean-pool $\oplus$ max-pool (192), classifier $192 \to 64 \to 2$
Optimiser	AdamW, lr 10^{-3}, weight decay 0.01, grad-clip 1.0
Scheduler	OneCycleLR, max lr 5×10^{-3}, pct-start 0.1
Loss	Cross-entropy, label smoothing 0
Schedule	≤ 150 epochs, batch 64, early stop on val F_1 (patience 20)
Calibration	LBFGS, max-iter 50, lr 0.01
XGBoost	200 estimators, max-depth 8, lr 0.1, subsample 0.8, colsample 0.8
Basic GCN	$3\times$ GCNConv, hidden 64, mean-pool readout, dropout 0.2
Seed	42 (split, sub-split, dataloader, torch RNG)

8. Results and Conclusions

8.1 Head-to-Head Comparison

Table 8.1 and Figure 8.1 report all three models on the same 11,880 shared test wallets. The GNN leads on every metric except precision, where it is within half a point of XGBoost while recovering *far* more criminals (recall 0.884 vs. 0.673).

Table 8.1. Test-set performance on identical wallets (n = 11,880 shared). Best per column in bold.

Model	F_1	Accuracy	Precision	Recall	ROC-AUC
OptimalGNN (ours)	**0.869**	**0.894**	0.855	**0.884**	**0.959**
XGBoost (tabular)	0.755	0.826	**0.861**	0.673	0.909
Basic GCN (graph baseline)	0.582	0.701	0.658	0.522	0.777

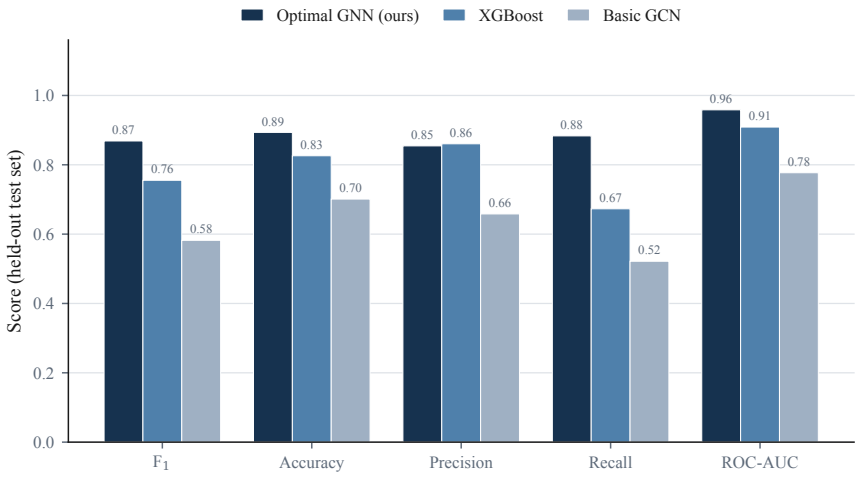

Figure 8.1. Metric-by-metric comparison of the three models on the shared held-out test set.

8.2 Training Dynamics

The GNN was trained for up to 150 epochs with early stopping on the validation F_1. Figure 8.2 shows training loss falling smoothly while validation F_1 plateaus; the best epoch (81) is selected and the remaining epochs add no validation gain, confirming the model is neither under- nor over-trained.

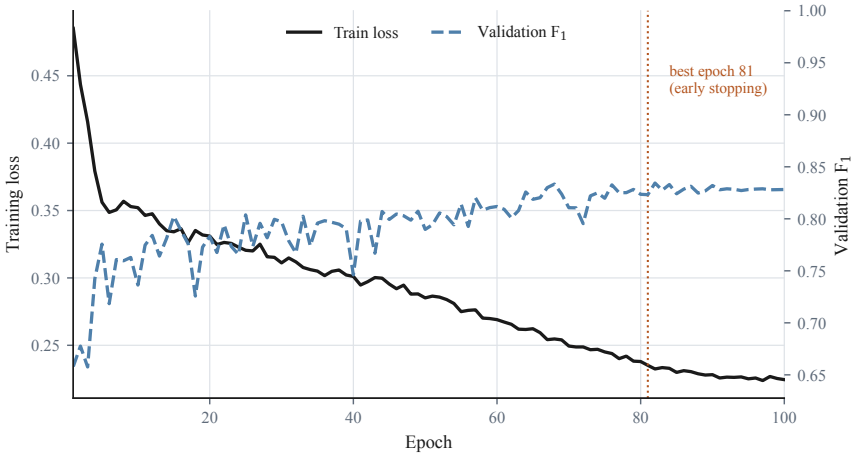

Figure 8.2. Training loss (left axis) and validation F_1 (right axis) over epochs. The dotted line marks the early-stopping / best epoch.

8.3 What the Gaps Mean

Two contrasts tell the story. **Basic GCN** $\rightarrow$ **OptimalGNN** ($+0.29$ F_1): per-transaction detail (amount, direction, timing) plus our architectural additions matter a great deal. **XGBoost** $\rightarrow$ **OptimalGNN** ($+0.11$ F_1, $+0.21$ recall): the one-hop neighbourhood carries information the wallet's own statistics do not capture — the project's central result, and evidence for a structural signature of illicit activity that holds across the two merged datasets.

8.4 Discrimination and Errors

The confusion matrices (Figure 8.3) contrast all three models on the same wallets. Our GNN shows a balanced error profile — 553 false negatives and 713 false positives out of 11,880 — missing relatively few criminals while keeping benign wallets mostly clear, whereas the plain GCN misses almost half of the criminal class. The ROC curves (Figure 8.4) give the GNN AUC $= 0.959$, well above XGBoost (0.909) and the plain GCN (0.777).

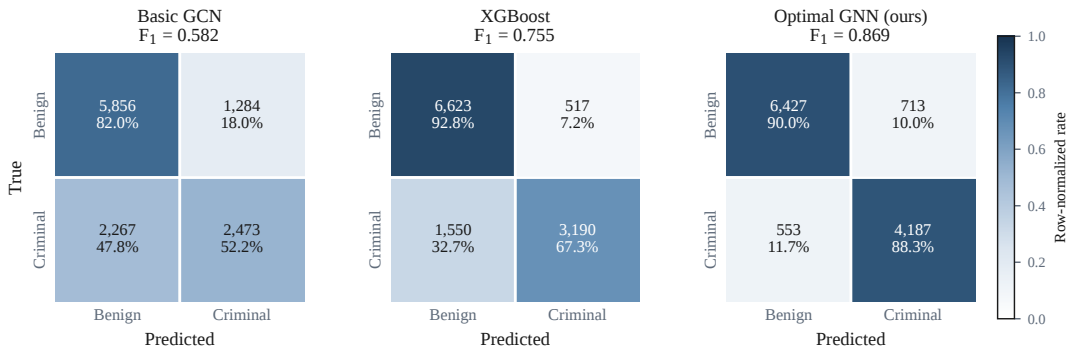

Figure 8.3. Confusion matrices on the shared held-out test set (colour is row-normalized; cells show counts and row percentages). Left to right: Basic GCN, XGBoost, and our Optimal GNN.

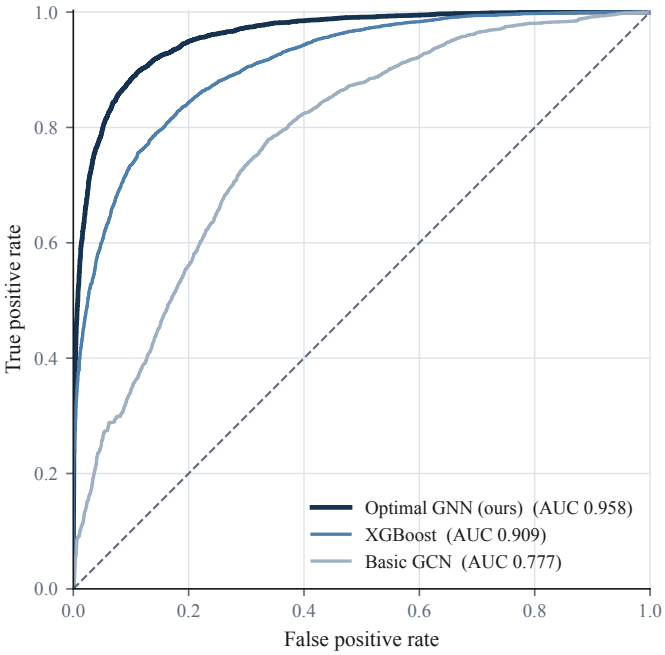

Figure 8.4. ROC curves for the three models on the held-out test set.

8.5 Calibration

After temperature scaling ($T = 1.06$), the predicted probabilities are usable as risk thresholds. The reliability diagram (Figure 8.5) tracks the diagonal closely, with a low Expected Calibration Error, so the risk score the web application displays can be trusted at face value rather than read only as a ranking.

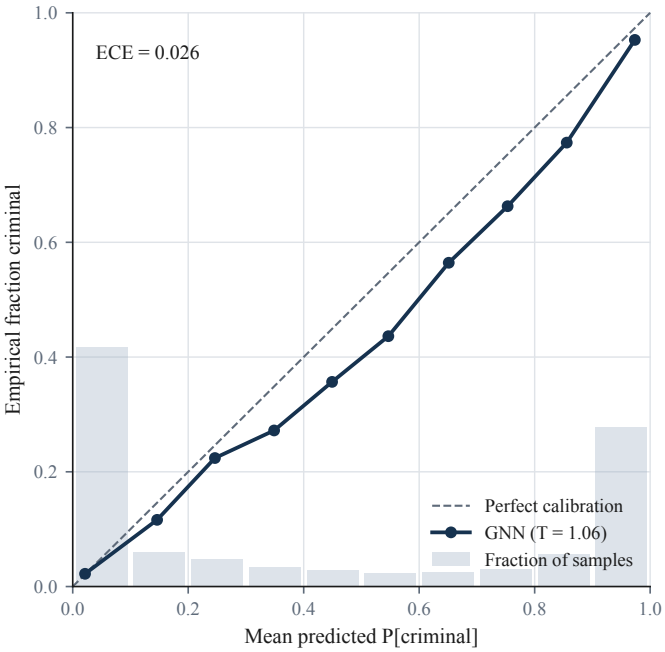

Figure 8.5. Reliability diagram for the GNN after temperature scaling; bars show the fraction of test samples in each confidence bin.

8.6 Interpretability

Gradient-saliency over the twelve inputs (Figure 8.6) is dominated by fee behaviour: `fee_per_tx` (0.63) and `fee_share_mean` (0.18) together account for the majority of attributed importance, with transaction-size range and total transactions next. This matches the EDA intuition that *how* a wallet spends on fees and structures its transactions, more than raw volume, separates criminal from benign behaviour.

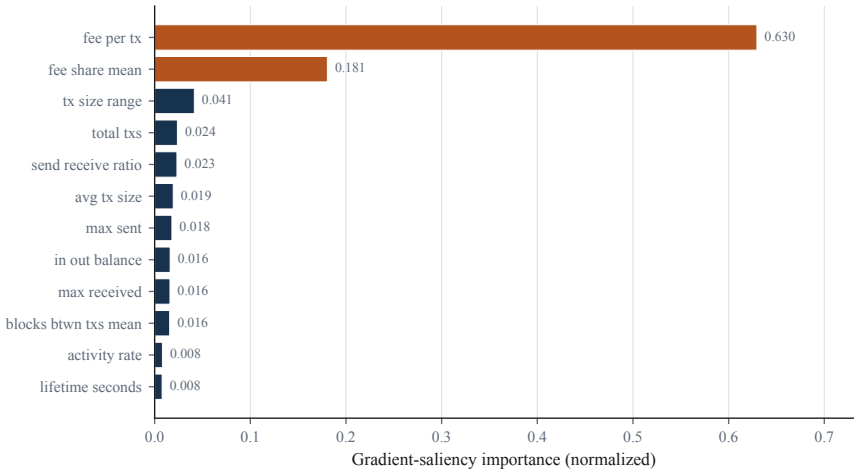

Figure 8.6. Per-feature gradient-saliency importance for the GNN (twelve log-scaled features).

9. Summary and Future Work

9.1 Summary

We built a graph-based Bitcoin wallet risk classifier that decides, from a single address and its public history, whether a wallet has been used for criminal activity. Trained on a balanced merge of REAL-CATS and Elliptic++, a three-layer GATv2 network reaches $F_1 = 0.869$ and ROC-AUC $= 0.959$ on a held-out test set, beating a tabular XGBoost baseline by $\sim$11 F_1 points and a plain GCN by $\sim$29. The controlled baselines show the gain comes specifically from reading the wallet's transaction neighbourhood, supporting the hypothesis that illicit activity has a structural signature that generalises across crime types and time periods. The model is deployed as a calibrated, live web application (cryptotrace.cs.bgu.ac.il).

9.2 Challenges

The main challenges were data-engineering rather than modelling: enforcing train/inference parity after a satoshi/BTC unit mismatch, keeping the probability range from collapsing (fixed by learnable ghost embeddings and by removing label smoothing, then temperature scaling), and building tens of thousands of ego-graphs against a rate-limited public API.

9.3 Future Work

Natural extensions are: (i) rebuild the training ego-graphs at full transaction pagination (current cached graphs captured only the first 25 transactions per wallet, a structure mismatch versus the 500-transaction inference path); (ii) train on the larger graph corpus once the full download completes; (iii) add an independent validation split for calibration instead of reusing the test set; (iv) build a live adversarial test set of recent wallets in neither source to stress-test generalisation via precision@k; and (v) explore temporal GNN extensions and focal loss (already implemented but unused) for the residual class-hard cases.

Bibliography

[1] Ismail Alarab and Simant Prakoonwit. Graph-based LSTM for illicit bitcoin transaction detection. *IEEE Access*, 10:41385–41397, 2022.

[2] S. Asiri, A. Alahmadi, R. Alsubaie, and A. Alqarni. Graph convolutional networks for illicit bitcoin transaction detection. *IEEE Access*, 13:11245–11258, 2025.

[3] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022.

[4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[5] Youssef Elmougy and Ling Liu. Demystifying fraudulent transactions and illicit nodes in the bitcoin network for financial forensics. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2023. Elliptic++ dataset. Available at `https://github.com/git-disl/EllipticPlusPlus`.

[6] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

[8] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[9] X. Ouyang, Y. Liu, Z. Zhang, and S. Wang. Subgraph representation learning for money laundering detection in cryptocurrency networks. *ACM Transactions on Knowledge Discovery from Data*, 18(2):1–23, 2024.

[10] RL-GNN Fusion Consortium. RL-GNN fusion for real-time fraud detection in imbalanced transaction streams. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 10234–10242, 2025.

[11] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. DropEdge: Towards deep graph convolutional networks on node classification. In *International Conference on Learning Representations (ICLR)*, 2020.

[12] J. Su et al. REAL-CATS: A benchmark dataset for modern cryptocurrency crime detection. Technical report, SJD/SEU, 2024. Expert-curated dataset of criminal and benign Bitcoin addresses. Available at https://github.com/sjdseu/Real-CATS.

[13] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[14] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*, 2019.

[15] Y. Zhang, H. Liu, X. Wang, J. Chen, and L. Zhao. GAT-ResNet: Attention-based graph neural networks for blockchain fraud detection. *Knowledge-Based Systems*, 278:110912, 2024.

A. Deployed System: Characterisation and Testing

A web application was built to *use* the model, so per the course guidelines it is characterised and tested here rather than in the main body.

A.1 Architecture

The system has two components. A **FastAPI** backend loads the trained weights (`outputs/gnn_model.pt`) and the temperature scaler, exposes an `/analyze` endpoint that, given an address, fetches its transactions from `mempool.space`, builds the ego-graph, runs inference, and returns a calibrated verdict plus supporting data. A **React + Vite** frontend lets a visitor paste an address and see the verdict, an explanation, and transaction context. The app is deployed on the BGU CS department servers at cryptotrace.cs.bgu.ac.il.

A.2 Functional Requirements

The user pastes a valid Bitcoin address; the system returns a criminal/benign verdict with a calibrated probability, the top contributing features, and a summary of the wallet's transaction history and balance. Invalid or empty addresses are rejected with a clear message.

A.3 Non-Functional Requirements

Analysis runs off the event loop so the UI stays responsive; wallet data is cached to avoid repeated API calls; transaction fetching is paginated to 500 transactions and issued sequentially to respect `mempool.space` rate limits; the UI meets basic accessibility and contrast guidelines (verdict shown above model internals).

A.4 Testing

Testing focused on model quality and behaviour (Chapters 7–8): held-out evaluation of all three models on identical wallets; calibration checks (Brier, ECE, reliability); per-source slicing to confirm neither REAL-CATS nor Elliptic++ carries the headline number alone; a misclassification dump (false positives / false negatives to CSV); and gradient-saliency interpretability. A planned extension is an automated unit-test suite and a live adversarial

precision@k set. Every evaluation artifact regenerates from the saved weights and the calibration scalar (seed 42).

A.5 Reproducibility

Repository. github.com/NehorayChalfon0166/final_project. **Live demo.** crypto-trace.cs.bgu.ac.il.

Run the full offline pipeline with:

```
python run_pipeline.py --all
```

or step by step: `--prepare` (build the balanced dataset) → `--graphs` (build/cache ego-graphs) → `--baseline` (XGBoost) → `--train` (GNN) → `--evaluate` (metrics, comparison, calibration, error dumps). A single seed (42) governs the dataset split, the validation sub-split, the dataloader shuffle, and the torch RNG, so every number in this report is reproducible from the saved model weights and the temperature scalar. Stack: PyTorch + PyTorch Geometric, XGBoost, FastAPI, React + Vite, and the `mempool.space` API.